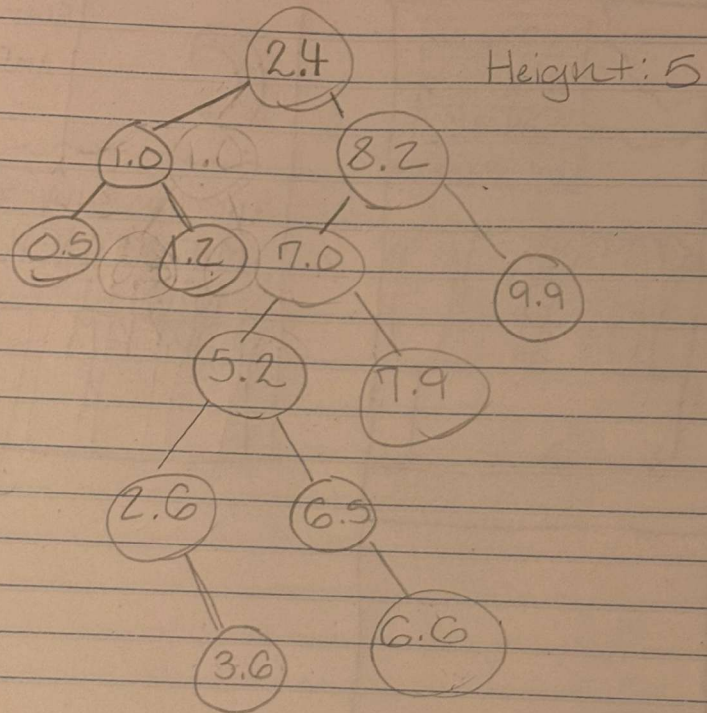


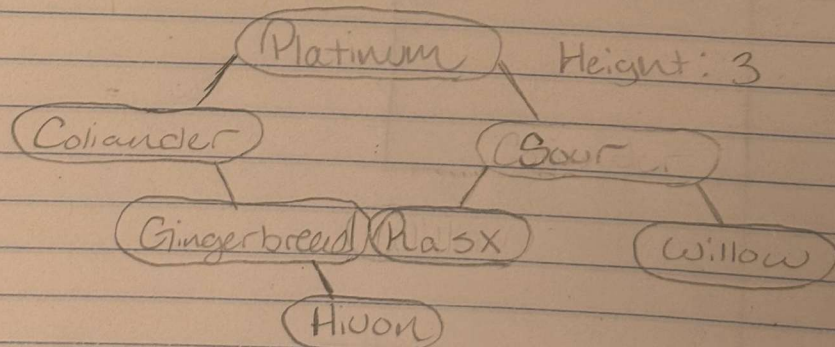
1.

Assignment 6 - Trees & Heaps

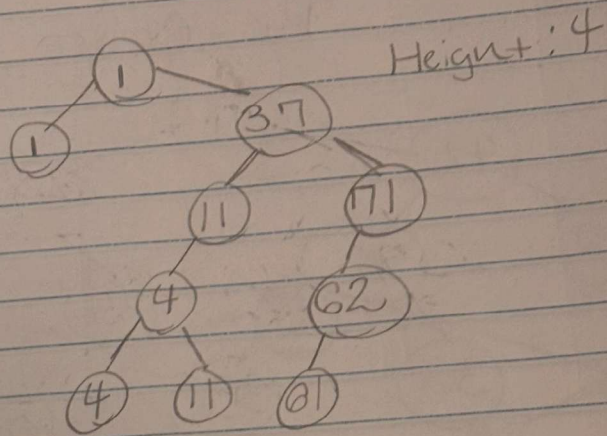
1) a)



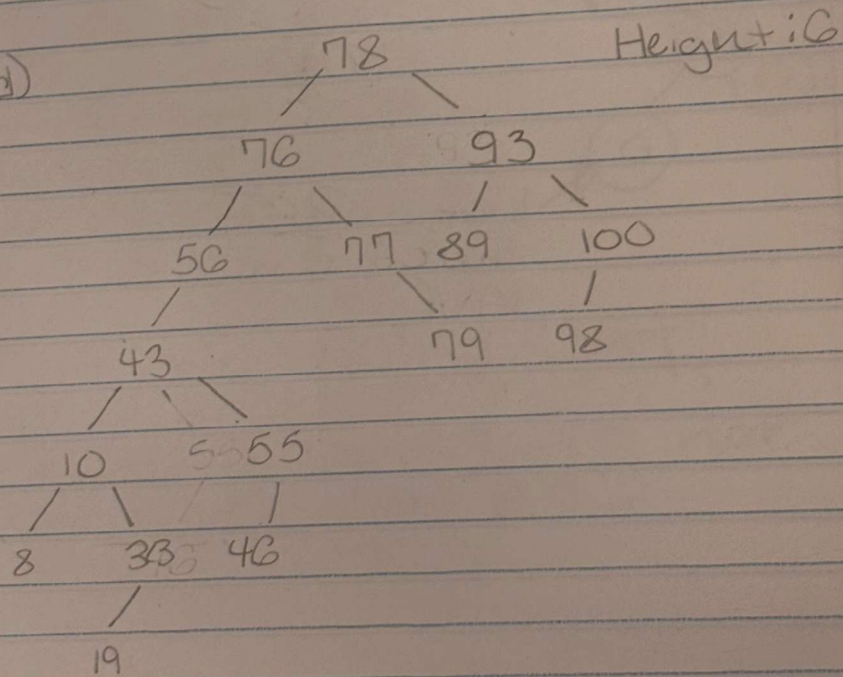
b)



c)



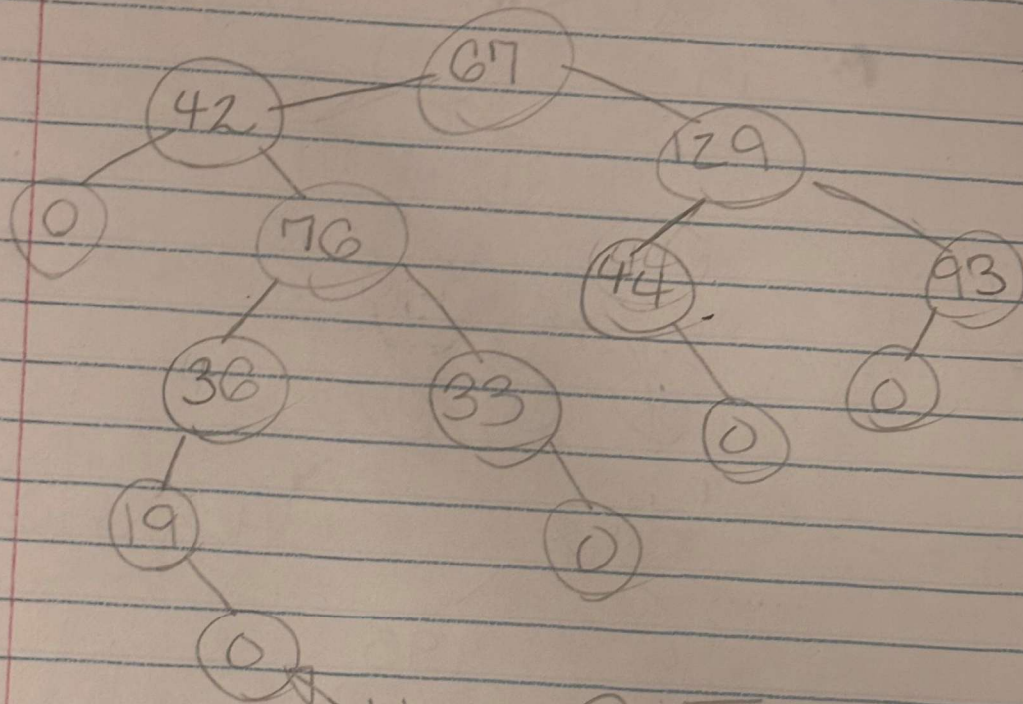
d)



2.

2) a)

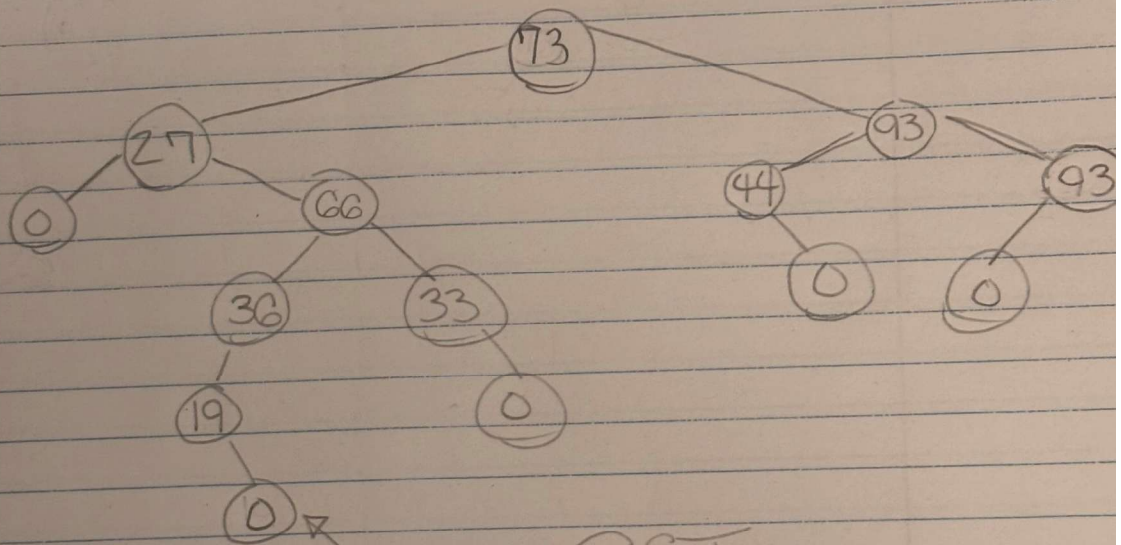
Pre-Order



Not a BST,
therefore not an
AVL

a)

In Order



Not a BST,
Therefore not
an AVL

3. Huffman and EncodingSystem.

In my Huffman, the time complexities go as followed:

- `FrequencyCount()` – $O(n)$ time, where n is the length of the string input, as with each letter, it checks that letter's frequency and adds it to the hashMap, which are all done in constant time operations. Space complexity is $O(k)$ for all existing letters in the string, as one frequency count is stored per value.
- `BuildHuffman()` – $O(k^3)$ time, $O(k)$ space
- `genCode()` – $O(k^3)$ time, $O(k)$ space
- `encode()` – $O(n)$ time, $O(n)$ space

4. Find Element in Encrypted Tree.

In all cases, The time complexity is $O(n)$, where n is the size of the array. In total, there are 4 methods that help in checking for an element: the main function exists, `TreeBuilder`, `assignValues`, and `search`. The `TreeBuilder` always checks and adds every value of the array to the tree. The `assignValues` functions checks all values before assigning values, which is all done through constant operations. The `Search` function has a worst case complexity of $O(n)$ if it has to visit every node, but the best case of $\Omega(n)$ if the root is the target. When it comes to the main function, which combines the other three, the complexity becomes $O(n) + O(n) + O(n)$ or $O(n) + O(n) + O(1) = O(n)$. In other terms, the average case is $\Theta(n)$. As for space complexity, that will always be $O(n)$, as all variables in the array are stored into a Tree.

5. AllLess

`TreeBuilder` had a complexity of $O(n)$, where n is the size of the array input, as the tree is built to store all values. Space complexity is $O(n)$.